

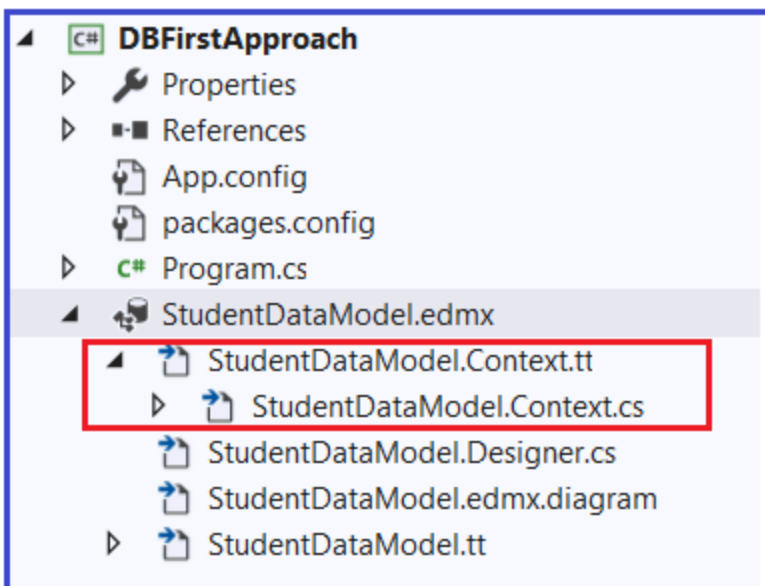
DbContext Class in Entity Framework Database First Approach

In this article, I am going to discuss **DbContext Class in Entity Framework Database First Approach** in detail with Examples. Please read our previous article where we discussed [Model Browser in Entity Framework](#). At the end of this article, you will understand the need and use of the Entity Framework DbContext class in detail. We are going to work with the same example that we created in our Introduction to Entity Framework Database First Approach article. Please read our introduction to [Entity Framework Database First](#) article before proceeding to this article.

Entity Framework DbContext Class:

According to [MSDN](#), a DbContext instance represents a combination of the Unit Of Work and Repository Patterns such that it can be used to query from a database and group together changes that will then be written back to the store as a unit. So, in simple words, we can say that the DbContext instance is the primary class that is responsible for interacting with the database and performing the database CRUD Operations.

Whenever we use Entity Framework Database First Approach to interact with an existing database, then the EDM (Entity Data Model) creates the context class which is inherited from the System.Data.Entity.DbContext class. Every Entity Data Model (EDM) generates one context class for the database. In order to verify this, expand the .EDMX file in the solution explorer and open **<EDM Name>.Context.tt** as shown in the below image. In our example, it will be StudentDataModel.Context.tt and inside this you will find the context class as shown in the below image. Here, the context class file name is StudentDataModel.Context.cs.



StudentDataModel.Context.tt:

This is the T4 template and this T4 template file generates a context class (in this case the context class file name is StudentDataModel.Context.cs), and whenever we change the Entity Data Model (.edmx file), then again the T4 template is also going to update the context class.

You can see the context class file by expanding **StudentDataModel.Context.tt**. The context class resides in the **<EDM Name>.Context.cs** file. In our case the context class file name is **StudentDataModel.Context.cs**. The default context class name is **<DB Name>Entities**. For example, the context class name for EF_Demo_DB is EF_Demo_DBEntities and is derived from the DbContext class as shown below.

```

namespace DBFirstApproach
{
    public partial class EF_Demo_DBEntities : DbContext
    {
        public EF_Demo_DBEntities()
            : base("name=EF_Demo_DBEntities")
        {
        }

        protected override void OnModelCreating(DbModelBuilder modelBuilder)
        {
            throw new UnintentionalCodeFirstException();
        }

        public virtual DbSet<Course> Courses { get; set; }
        public virtual DbSet<Standard> Standards { get; set; }
        public virtual DbSet<Student> Students { get; set; }
        public virtual DbSet<StudentAddress> StudentAddresses { get; set; }
        public virtual DbSet<Teacher> Teachers { get; set; }
        public virtual DbSet<vwStudentCourse> vwStudentCourses { get; set; }

        public virtual int spDeleteStudent(Nullable<int> studentId) {---}
        public virtual ObjectResult<spGetCoursesByStudentId_Result> spGetCoursesByStudentId(Nullable<int>
studentID)
        {-----}
        public virtual ObjectResult<Nullable<decimal>> spInsertStudent(Nullable<int> standardId, string
firstName, string lastName)
        {-----}
        public virtual int spUpdateStudent(Nullable<int> studentId, Nullable<int> standardId, string firstName,
string lastName)
        {-----}
    }
}

```

Context class must be inherited from DbContext class

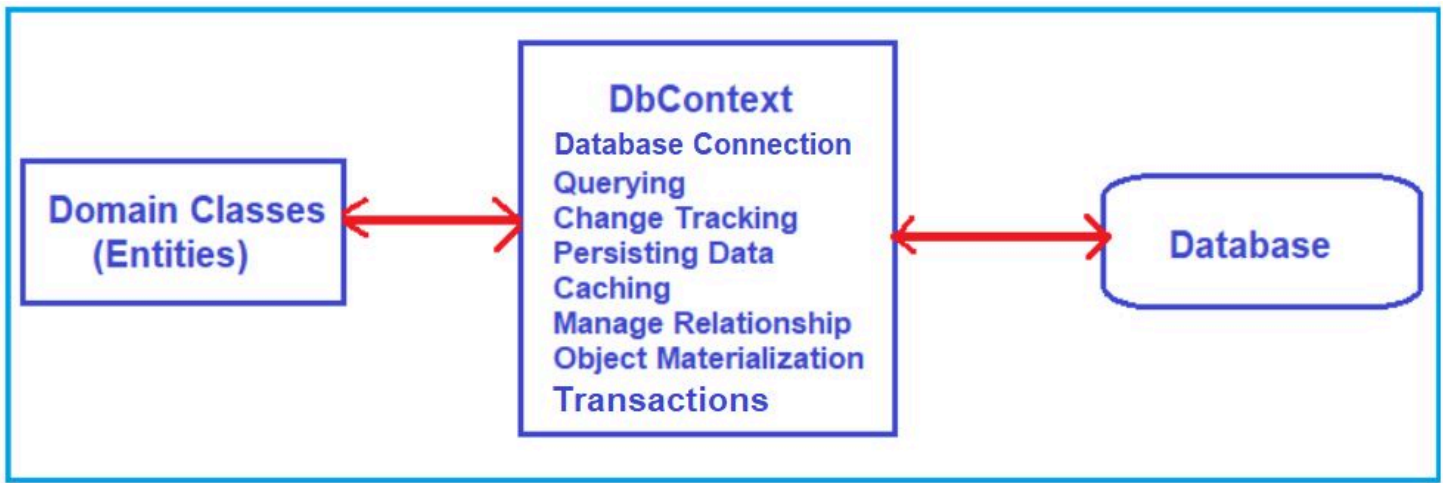
Fluent API

Entity Set

The class is derived from the **System.Data.Entity.DbContext** class is called a context class in the entity framework. As you can see, this context class contains one constructor, properties of **DbSet<TEntity>** type for each database table and views, methods for each stored procedure and table-valued functions, and one method for Fluent API i.e. **OnModelCreating**. Now, let us proceed and try to understand what are the responsibilities of the Entity Framework Context class in detail.

What are the responsibilities of DbContext Class in Entity Framework?

The DbContext Class in Entity Framework is an important class and as we discussed, it is primarily responsible for interacting with databases and performing the database operations like **SELECT**, **INSERT**, **UPDATE**, and **DELETE** and also executing the Stored Procedures. That means it acts as a bridge between your domain classes or entity classes and the underlying relational database. For a better understanding, please have a look at the following image which shows the responsibilities of DbContext class in Entity Framework.



Let us understand what are the responsibilities done by DbContext class in Entity Framework. The DbContext class is responsible for the following activities in Entity Framework:

1. **Manage Database Connection:** Entity Framework is built on top of the ADO.NET, and Entity Framework also manages a Connection Pool. DbContext opens the connection whenever needed at the time of communication with the underlying relational Database. As it is managing the Connection pool internally, as a developer we need not be worried about the connections.
2. **Querying:** Mostly when we are working with Entity Framework, we used Linq Queries. The DbContext class is responsible for converting the LINQ-to-Entities queries to SQL query which is going to be understood by the underlying database and then sends them to the database for execution.
3. **Change Tracking:** In our [Entity States in Entity Framework](#) article we discussed the different states of an Entity and it is also possible to change the state from one state to another state. The Entity Framework DbContext class is also responsible for keeping track of changes that occurred on the entities after querying from the database. And based on the entity state, when we called the SaveChanges method, the required SQL statements are going to be generated and executed in the database. For the Unchanged and Detached State, no SQL statement is going to be generated and executed. And one more point, once an entity moves to Detached State, then the context class is not going to keep track of that entity anymore.
4. **Persisting Data:** The DbContext class is also responsible for performing the Insert, Update and Delete operations to the database which is persistent data permanently to the database. Based on the Entity State, the context is going to be generated the required SQL statement. For example, if the Entity state is Added, then it is going to generate and execute an INSERT SQL statement in the database. If the Entity state is Modified, then it is going to generate and execute an UPDATE SQL statement in the database. And if the Entity state is Deleted, then it is going to generate and execute a DELETE SQL statement in the database.
5. **Caching:** Nowadays caching plays an important role in almost all types of applications to improve performance. By default, the DbContext class provides the first-level cache. That means it stores the entities which have been retrieved from the database during the lifetime of a context class. That means the repeated querying will return data from the cache instead of hitting the database again and again which will reduce the number of round trips to the data which will ultimately improve the overall performance of the application.
6. **Manage Relationship:** It Manages the relationships among the entities using CSDL, SSDL, and MSL in Db-First or Model-First approach, and using fluent API configurations in the Code-First approach. That means which entity is mapped with which database table, which property is mapped with which database table columns, and what is the relationship between the entities i.e. one-to-one or one-to-many or one-to-many, etc, are managed by the context class.
7. **Object Materialization:** The DbContext is responsible for converting the raw data retrieved from the database into entity objects in Entity Framework.

Note: If you are not understanding the above points then don't worry, we will discuss all the above points in detail with examples in our upcoming article.

Entity Framework DbContext class:

Now, let us proceed and try to understand the auto-generated code of our content class. The following EF_Demo_DBEntities class (this is our context class as it is derived from the DbContext class) is an auto-generated

class by the Entity Framework for the EF_Demo_DB database.

```
//-----  
// <auto-generated>  
//     This code was generated from a template.  
//  
//     Manual changes to this file may cause unexpected behavior in your application.  
//     Manual changes to this file will be overwritten if the code is regenerated.  
// </auto-generated>  
//-----  
namespace DBFirstApproach  
{  
    using System;  
    using System.Data.Entity;  
    using System.Data.Entity.Infrastructure;  
    using System.Data.Entity.Core.Objects;  
    using System.Linq;  
  
    public partial class EF_Demo_DBEntities : DbContext  
    {  
        public EF_Demo_DBEntities()  
            : base("name=EF_Demo_DBEntities")  
        {  
        }  
  
        protected override void OnModelCreating(DbModelBuilder modelBuilder)  
        {  
            throw new UnintentionalCodeFirstException();  
        }  
  
        public virtual DbSet<Course> Courses { get; set; }  
        public virtual DbSet<Standard> Standards { get; set; }  
        public virtual DbSet<Student> Students { get; set; }  
        public virtual DbSet<StudentAddress> StudentAddresses { get; set; }  
        public virtual DbSet<Teacher> Teachers { get; set; }  
        public virtual DbSet<vwStudentCourse> vwStudentCourses { get; set; }  
  
        public virtual int spDeleteStudent(Nullable<int> studentId)  
        {  
            var studentIdParameter = studentId.HasValue ?  
                new ObjectParameter("StudentId", studentId) :  
                new ObjectParameter("StudentId", typeof(int));  
  
            return  
                ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction("spDeleteStudent",  
                    studentIdParameter);  
        }  
  
        public virtual ObjectResult<spGetCoursesByStudentId_Result>  
            spGetCoursesByStudentId(Nullable<int> studentID)  
        {  

```

```

        var studentIDParameter = studentID.HasValue ?
            new ObjectParameter("StudentID", studentID) :
            new ObjectParameter("StudentID", typeof(int));

        return
        ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction<spGetCoursesByStu
("spGetCoursesByStudentId", studentIDParameter);
    }

    public virtual ObjectResult<Nullable<decimal>> spInsertStudent(Nullable<int>
standardId, string firstName, string lastName)
    {
        var standardIdParameter = standardId.HasValue ?
            new ObjectParameter("StandardId", standardId) :
            new ObjectParameter("StandardId", typeof(int));

        var firstNameParameter = firstName != null ?
            new ObjectParameter("FirstName", firstName) :
            new ObjectParameter("FirstName", typeof(string));

        var lastNameParameter = lastName != null ?
            new ObjectParameter("LastName", lastName) :
            new ObjectParameter("LastName", typeof(string));

        return
        ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction<Nullable<decimal>>
("spInsertStudent", standardIdParameter, firstNameParameter, lastNameParameter);
    }

    public virtual int spUpdateStudent(Nullable<int> studentId, Nullable<int>
standardId, string firstName, string lastName)
    {
        var studentIdParameter = studentId.HasValue ?
            new ObjectParameter("StudentId", studentId) :
            new ObjectParameter("StudentId", typeof(int));

        var standardIdParameter = standardId.HasValue ?
            new ObjectParameter("StandardId", standardId) :
            new ObjectParameter("StandardId", typeof(int));

        var firstNameParameter = firstName != null ?
            new ObjectParameter("FirstName", firstName) :
            new ObjectParameter("FirstName", typeof(string));

        var lastNameParameter = lastName != null ?
            new ObjectParameter("LastName", lastName) :
            new ObjectParameter("LastName", typeof(string));

        return
        ((IObjectContextAdapter)this).ObjectContext.ExecuteFunction("spUpdateStudent",
studentIdParameter, standardIdParameter, firstNameParameter, lastNameParameter);
    }

```

```

    }
}
}

```

As you can see in the above code, the context class (EF_Demo_DBEntities) includes the entities as DbSet<TEntity> type for all the tables and views. In our next article, we will discuss the DbSet class in detail. The point that you need to remember is, whenever a class is included as DbSet<TEntity> type in the context, then it is considered an entity and then we can access this entity using the context object.

It also includes functions for the stored procedures and table-valued functions. In our example, we don't have any table-valued functions but we have stored procedures, and it creates separate functions for each stored procedure. The OnModelCreating method allows us to configure the model using the DbModelBuilder Fluent API in Entity Framework and we will discuss it in detail when we discuss the Code First Approach of Entity Framework.

DbContext Methods in Entity Framework:

Now, let us proceed further and try to understand the important methods provided by the DbContext class in Entity Framework.

1. **Entry(object entity):** It is used to get a DbEntityEntry object for the given entity providing access to information about the entity and the ability to perform actions on the entity.
2. **SaveChanges:** Saves all changes made in the context object to the underlying database. That means, it executes INSERT, UPDATE, and DELETE commands to the database for the entities with Added, Modified, and Deleted states.
3. **SaveChangesAsync:** This method is used to asynchronously saves all changes made in this context to the underlying database. That means, it executes INSERT, UPDATE, and DELETE commands to the database for the entities with Added, Modified, and Deleted states asynchronously.
4. **Set(Type entityType):** This method returns a non-generic DbSet instance for access to entities of the given type in the context and the underlying database. Here, the parameter entityType specifies the type of entity for which a set should be returned. That means it creates a DbSet<TEntity> that can be used to query and save instances of TEntity into the database.
5. **OnModelCreating:** This method is called when the model for a derived context has been initialized, but before the model has been locked down and used to initialize the context. The default implementation of this method does nothing, but it can be overridden in a derived class such that the model can be further configured before it is locked down.

Entity Framework DbContext Class Properties

1. **ChangeTracker { get; }:** This property provides access to features of the context that deal with change tracking of entities. This is a read-only property.
2. **Configuration { get; }:** This property provides access to configuration options for the context. This is a read-only property.
3. **Database { get; }:** This property creates a Database instance for the context object that allows for creation/deletion/existence checks for the underlying database. This is a read-only property.

Example to Understand How to Perform CRUD Operations using DbContext Class in C#:

Using the Entity Framework DbContext class we can perform the database CRUD operations i.e. SELECT, INSERT, UPDATE, and DELETE Operations. Let us proceed and see how we can perform the database CRUD operations using DbContext class in Entity Framework.

Adding New Entity in Entity Framework Database First Approach:

Now, we need to add one entity i.e. one Student into the Student table. Adding a new object with the Entity Framework is very simple. Just create an instance of the entity and then register it using the Add method on the DbSet instance and

then call the SaveChanges method. The following piece of code in the main method does the same. So, please modify the main method of the Program class as shown below. The following code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Creating a new student
                var student = new Student
                {
                    FirstName = "Hina",
                    LastName = "Sharma",
                    StandardId = 1
                };
                //Adding new student to the context object
                //Now the context object will track this entity
                context.Students.Add(student);
                //Now the context object will track the Student entity as in
                Added State
                Console.WriteLine($"Before SaveChanges Entity State:
                {context.Entry(student).State}");
                //Calling the SaveChanges method to store the newly added
                entity into the database
                //The context object will generate the INSERT SQL Query as
                the Entity in Added State
                context.SaveChanges();
                //Once the SaveChanges Method Executed Successfully, the
                entity state will changed to Unchanged
                Console.WriteLine($"After SaveChanges Entity State:
                {context.Entry(student).State}");

                //Verifying the newly Added Student
                List<Student> listStudents = context.Students.ToList();
                foreach (Student std in listStudents)
                {
                    Console.WriteLine($"StudentID: {std.StudentId} Name =
                    {std.FirstName} {std.LastName}");
                }
                Console.ReadKey();
            }
        }
    }
}
```

Output: You can see the newly added student in the console window and you can also verify the same in the Student table of the EF_Demo_DB database.

```
Before SaveChanges Entity State: Added
After SaveChanges Entity State: Unchanged
StudentID: 1 Name = Virat Kohli
StudentID: 2 Name = Rohit Sharma
StudentID: 3 Name = Lokesh Rahul
StudentID: 4 Name = Smriti Mandana
StudentID: 5 Name = Hina Sharma
```



Newly Added Student

In the above example, when we add the student entity into the context object, it will be in the Added state and when we call the SaveChanges method, the context object will generate and execute the following INSERT SQL Script. Once the SaveChanges method execution is completed successfully, it will move the Entity state from Added to Unchanged state.

```
exec sp_executesql N'INSERT [dbo].[Student]([FirstName], [LastName], [StandardId])
VALUES (@0, @1, @2)
SELECT [StudentId]
FROM [dbo].[Student]
WHERE @@ROWCOUNT > 0 AND [StudentId] = scope_identity()',N'@0 varchar(100),@1 varchar(100),@2
int',@0='Hina',@1='Sharma',@2=1
```

Updating Existing Entities in Entity Framework Database First Approach using Context Object

Now, we need to update one existing entity i.e. we need to update the First name and Last name of the newly created student i.e. the student whose student id is 5. The following piece of code in the main method exactly does the same. So, please modify the main method of the Program class as shown below. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //First fetch the student data
                var student = context.Students.FirstOrDefault(x =>
x.StudentId == 5);
                //Here, as you fetch an existing record from the database
                using the context object
```

```

        //so context object will track that student entity
        //Here, the entity state is going to be Unchanged
        Console.WriteLine($"Retrieved Entity State:
{context.Entry(student).State}");

        //if the student exists then modify the data
        if (student != null)
        {
            //Once you update any of the properties of an existing
entity
            //the State will moved to Modified State
            student.FirstName = "Hina1";
            student.LastName = "Sharma1";
            Console.WriteLine($"Modified Entity State:
{context.Entry(student).State}");
            //Once you call the SaveChanges method, it will generate
and Execute UPDATE SQL Statement
            context.SaveChanges();
            //Once the SaveChanges executed Successfully, the Entity
State will moved to Unchanged
            Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(student).State}");
        }

        //Verifying the updated data in the database
        List<Student> listStudents = context.Students.ToList();
        foreach (Student std in listStudents)
        {
            Console.WriteLine($"StudentID: {std.StudentId} Name =
{std.FirstName} {std.LastName}");
        }
        Console.ReadKey();
    }
}
}
}
}

```

Output: You can see the updated data in the console window and you can also verify the same in the Student table of the EF_Demo_DB database.

```

Retrieved Entity State: Unchanged
Modified Entity State: Modified
After SaveChanges Entity State: Unchanged
StudentID: 1 Name = Virat Kohli
StudentID: 2 Name = Rohit Sharma
StudentID: 3 Name = Lokesh Rahul
StudentID: 4 Name = Smriti Mandana
StudentID: 5 Name = Hina1 Sharma1

```

In the above example, when we have updated the existing student entity. Once we update at least one property, the entity will be moved to the Modified state and when we call the SaveChanges method, the context object will generate

and execute the following UPDATE SQL Script. Once the SaveChanges method execution is completed successfully, it will move the Entity state from Modified to Unchanged state.

```
exec sp_executesql N'UPDATE [dbo].[Student]
SET [FirstName] = @0, [LastName] = @1
WHERE ([StudentId] = @2)
',N'@0 varchar(100),@1 varchar(100),@2 int',@0='Hina1',@1='Sharma1',@2=5
```

Deleting an Entity in Entity Framework Database First Approach using Context Object

Now, we need to delete the newly created entity i.e. the student whose student id is 5. To delete an entity using Entity Framework, we need to use the Remove method on the DbSet object. The Remove method works for both existing and newly added entities. Calling Remove on an entity that has been added but not yet saved to the database will cancel the addition of the entity. The entity is removed from the change tracker and is no longer tracked by the DbContext object and in that case, there is no database interaction when we call the SaveChanges method. But, calling the Remove method on an existing entity will mark that entity state to be Deleted and when we call the SaveChanges method then the context object will generate and execute the DELETE SQL Command and once the execution of the SaveChanges method is completed, it will move the Entity State from Deleted to Detached state which means it is no longer tracked by the context object. In the following example, the student is removed from the database whose id is 5. The following code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //First fetch the student data
                var student = context.Students.FirstOrDefault(x =>
x.StudentId == 5);
                //At this point the Entity State is Unchanged
                Console.WriteLine($"Before Deletting Entity State:
{context.Entry(student).State}");
                //if the student exists then remove the student
                if (student != null)
                {
                    //The Remove Method will mark the Entity State as Deleted
                    context.Students.Remove(student);
                    Console.WriteLine($"After Deletting Entity State:
{context.Entry(student).State}");
                    //The SaveChanges Method will generate and execute the
                    DELETE SQL Statement
                    context.SaveChanges();
                    //Now, the Entity State is moved to Detached and no
                    longer tracked by the Context object
                    Console.WriteLine($"Before Deletting Entity State:
{context.Entry(student).State}");
                }
            }
        }
    }
}
```

```

        //Verifying the updated data in the database
        List<Student> listStudents = context.Students.ToList();
        foreach (Student std in listStudents)
        {
            Console.WriteLine($"StudentID: {std.StudentId} Name = 
{std.FirstName} {std.LastName}");
        }
        Console.ReadKey();
    }
}
}
}
}

```

Output: Now, you can observe the deleted data not showing in the Console. You can also verify the same in the Student table of the EF_Demo_DB database.

```

Before Deletiiing Entity State: Unchanged
After Deletiiing Entity State: Deleted
Before Deletiiing Entity State: Detached
StudentID: 1 Name = Virat Kohli
StudentID: 2 Name = Rohit Sharma
StudentID: 3 Name = Lokesh Rahul
StudentID: 4 Name = Smriti Mandana

```

In the above example, when we deleted one existing student entity. Once we remove the entity, the entity will be moved to the Deleted state and when we call the SaveChanges method, the context object will generate and execute the following DELETE SQL statement. Once the SaveChanges method execution is completed successfully, it will move the Entity state from Deleted to Detached state and no longer be tracked by the context object.

In the next article, I am going to discuss the [DbSet in Entity Framework](#) with Examples. Here, in this article, I try to explain the **DbContext Class in Entity Framework Database First Approach** and I hope you enjoyed DbContext class in the Entity Framework article. Please give your valuable feedback and suggestions about this article.